Reducing Notification Fatigue using a Portable Adaptive-Baseline PM2.5 Monitor for Persistently Polluted Cities

***Abstract*—Air pollution in Dhaka is severe and it almost never lets up. Conventional monitors alert the user whenever PM2.5 crosses a fixed limit. In Dhaka that limit is crossed nearly all day. The alerts stop meaning anything and are eventually ignored. To address this, a portable monitor was built around an ESP32-C3 microcontroller, a PMS7003 particle sensor and a BME280, all on one perfboard running from a single 18650 cell. It does not rely on a fixed limit, and instead the recent PM2.5 history is tracked and an alert is raised only when a reading rises sharply above that baseline. This rule, Adaptive Baseline Spike Detection (ABSD), is specified with pseudocode and all of its parameters. Alerts are delivered over Bluetooth Low Energy to a React Native app backed by Firebase. The device was carried by one of the authors for two weeks, one week per mode. In fixed-threshold mode the phone received dozens of alerts every day. Spike mode produced only a few, and they were considerably more meaningful. This suggests adaptive alerting can reduce alert fatigue even where the air never gets clean.**

***Index Terms*—Alert fatigue, adaptive thresholds, PM2.5, personal sensing, wearable monitor, Dhaka**

I. Introduction

Dhaka has been known for years as a city with some of the worst air anywhere. In winter the daily $PM_{2.5}$ level usually sits somewhere between 150 and 200 μg m^{-3}, and the WHO 24-hour guideline is only 15 μg m^{-3} [1]. Being exposed to air like this over a long period is harmful for health, and it cannot be overstated how serious that is [2].

Now, most air quality monitors work in a fairly simple way. The user is alerted whenever $PM_{2.5}$ goes over a set limit. In Dhaka the concentration is above that limit for most of the day, therefore the alerts keep coming at home, on the bus, in class and everywhere in between. After a while people just stop paying attention to them. This is known as notification fatigue. Telling someone in Dhaka that the air is bad does not tell them anything they did not already know. What they actually need to know is if the air around them has suddenly gotten worse than it was a few minutes ago. Say the reading jumps from 150 to 190. This usually means something nearby is causing it, like construction work or a diesel bus idling on the side of the road. A fixed threshold however treats it the same as the background.

In the proposed system, the recent $PM_{2.5}$ readings are used as a personal baseline instead, and an alert is only sent if the current reading goes well above that.

The key contributions of this study are as follows:

1) A pocket-sized monitor intended for cities where background $PM_{2.5}$ remains high throughout the day.
2) An alert rule called Adaptive Baseline Spike Detection (ABSD), written out formally and as pseudocode, with the reason behind each parameter explained.
3) Three design goals from alert fatigue research, and a low-cost perfboard build made to meet them.
4) Two weeks of daily commuting in Dhaka by a single user, used to compare fixed-threshold and spike-based alerts.

The rest of this paper goes as follows. Related work is reviewed in Section II. Section III explains the design goals and the hardware, while the firmware, the app and the backend are covered in Section IV. The field test is reported in Section V, the findings and their limits are discussed in Section VI and Section VII concludes the paper.

II. Literature Review

A. Choice of Sensor

Since it has been tested against reference instruments before, the PMS7003 was picked for this project. In a study by Badura *et al.* [3], $R^2 > 0.9$ was found for $PM_{2.5}$ trends however the sensor read too high when the humidity was high. It was later shown by the same group that it can be used for tracking trends as long as it is calibrated from time to time [4]. A review of low-cost PM sensors was done by Alfano *et al.* [5] as well. Calibration and the differences from one unit to another were still listed there as open problems. Those limitations matter less here. Relative spikes are detected, not absolute concentrations.

B. Portable Monitors that Report to the User

Portable monitors that push readings to a phone are not new. Most of that work is judged on measurement accuracy. Whether the alert logic suits the environment is rarely asked.

Alvear-Puertas *et al.* fitted a vehicle with MQ-series gas sensors and an ESP32, sending the data over MQTT so that classification could run on a remote server [6]. The results were good. However the system was tied to the vehicle and its alerts were threshold-based. SentinAir, built by Suriano, offered a display and WiFi remote control but lacked a smartphone app and any form of adaptive alerting [7]. Sharma and Kumar streamed ESP32 readings to a phone through Blynk, with notifications still triggered by fixed limits [8]. Haque *et al.* combined a PMS7003 with a web dashboard intended for developing countries [9]. The alerts were threshold-based as well. Palomeque-Mangut *et al.* linked a wearable to a phone

over BLE and reported 97% accuracy in controlled tests [10]. Nguyen *et al.* put $PM_{2.5}$ and CO_2 readings on an Apple Watch and sent push alerts from it, but here too the alerts were triggered by fixed values [11].

Sending data to a phone has been done many times already. That is not where the gap is. What none of these systems do is decide whether something deserves an alert by looking at what the user was breathing a few minutes earlier. In a city where the baseline never really goes down, that is the part that matters the most.

C. Alert Fatigue in Personal Sensing

Alert fatigue is something that public health research has been looking at for years. However it has not been looked at much in air quality monitoring. Baseman *et al.* showed that frequent, unprioritised notifications lead to desensitisation over time [12]. Mehrotra *et al.* showed that learning notification preferences from context cuts interruptions while keeping the alerts that matter [13]. None of this has been carried into air quality monitors. However static thresholds are still the default, even in places where the threshold is exceeded the moment a person steps outside.

Becker *et al.* went through 33 studies on personal exposure feedback. The effect on behaviour was mixed, and participants often reported low self-efficacy [14]. Robinson *et al.* handed portable sensors to nineteen people and followed up with focus groups, where missing data and an air quality index that was hard to read turned out to be the main reasons people stopped [15]. Bernasconi *et al.* asked 16 participants to carry a PM monitor on a walk in Milan. Awareness increased. However so did a sense of powerlessness, since the readings kept changing and no clear action was available [16]. Feedback is only useful when it can be acted on. That is the gap this paper sits in.

D. Adaptive Thresholds and Change Detection

Detecting change in a signal is an old statistical problem. Page proposed the cumulative sum (CUSUM) scheme, in which small deviations from a reference level are accumulated and an alarm is raised once the running total becomes too large [17]. Roberts introduced the exponentially weighted moving average (EWMA) chart, where recent samples are weighted more heavily than older ones [18]. Both were designed for industrial process control. Chandola *et al.* surveyed anomaly detection more widely and described statistical methods as fitting a model of normal behaviour and flagging whatever that model considers unlikely [19], and ABSD belongs to that family. It is a moving-window z-score test with a floor on the spread, a minimum absolute rise and a debounce step. However CUSUM and EWMA assume a process that should stay at a fixed target. Commuters have no fixed target. So a short window and a manual reset were preferred over a long memory.

E. Summary of Related Systems

Table I summarises the related portable systems, with this work in the final row.

Previous systems do not combine pocket-sized hardware, phone-based feedback and alerting that adapts to the exposure of the individual user. For a city like Dhaka that combination is exactly what is missing. This paper proposes ABSD to provide it.

III. Design Goals and Hardware

Three design goals were derived from the review. DG1 requires alerts to depend on change relative to the user's recent exposure rather than on an absolute limit. DG2 requires the device to be small and cheap enough to be carried every day. DG3 requires that the baseline can be corrected without taking out the phone and that past events can still be looked at later in the app by the user.

A. Components and Assembly

An ESP32-C3 SuperMini was used as the controller. Its small footprint and on-chip BLE meant that no extra radio module had to be added. The PM readings come from a PMS7003. It is a laser particle counter. Temperature, humidity and pressure are measured by a GY-BME280 breakout board. All of it was soldered onto a double-sided 5 cm $\times$ 7 cm perfboard. The wires were kept as short as they could be. This helps with DG2. In Figure 1 the device can be seen while it is running, the red LEDs on the controller and the MT3608 boost converter are both on.

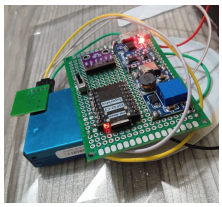


Fig. 1. Inside view of the device, with the PMS7003 (blue), the ESP32-C3 (bottom centre), the BME280 (purple, upper left) and the MT3608 (right).

B. Power Supply

One 18650 lithium-ion cell rated at 3.7 V is used to power the device. Charging is done through a TP4056 module. The same module keeps the cell safe from overcharging and deep discharge.

Beyond the switch the supply is split into two paths. Path A is stepped up to 5 V by an MT3608 converter and is used for the PMS7003 alone. The fan and laser inside the sensor need a stable 5 V, without it the fan slows and the readings become unreliable. Path B enters the RAW pin of the ESP32-C3, whose onboard regulator produces 3.3 V for the BME280. All grounds are tied together. UART and I2C both need a common reference, and without one the readings turn noisy and the microcontroller can restart. The complete wiring of the device is shown in Figure 2.

TABLE I
PORTABLE AIR QUALITY MONITORS THAT GIVE REAL-TIME FEEDBACK TO THE USER.

Ref.	Type	Hardware and Method	Feedback Channel	Main Result	Weakness
[6]	Portable IoT on a vehicle	MQ sensors, ESP32, MQTT, SVM in the cloud	Cloud dashboard	ML was feasible, over 90% accuracy	Tied to a vehicle, fixed alert thresholds
[7]	SentinAir field unit	Battery, built-in display, WiFi, MOx sensors	Display on the unit	Sensors compared in real conditions	No phone app, alerts not adaptive
[8]	IoT with a Blynk server	ESP32, several sensors, Blynk cloud	Blynk mobile app	Cheap remote viewing	Uses fixed pollutant limits
[9]	Low-cost IoT for developing regions	ESP32, PMS7003, web dashboard	Web page	Remote monitoring worked	Fixed thresholds, too big for a pocket
[10]	BLE wearable	MOS and PM sensors, BLE, phone app	Phone app	97% in the lab, 82.3% in the field	Threshold alerts, nothing adaptive
[11]	Air quality on a smartwatch	Apple Watch, $PM_{2.5}$/CO_2 sensors, phone app	Watch screen, push alerts	Quick glances raised awareness	Fixed triggers, few users
This work	Portable spike detection	ESP32-C3 with PMS7003 and BME280, BLE link, Firebase, React Native app	Phone app, web dashboard	Spike mode sent a few alerts a day, fixed mode sent dozens	First deployment, no humidity correction yet

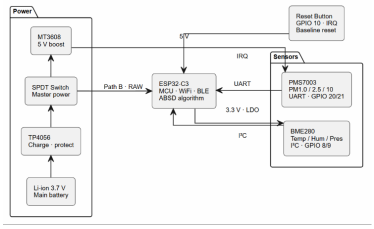


Fig. 2. Block diagram of the hardware, covering power, sensor wiring and the processing unit.

C. Baseline Reset Button

A tactile button was also added on GPIO10, with a pull-up resistor, for resetting the baseline. When someone walks out of an air-conditioned room with a $PM_{2.5}$ of around 20 and onto a street with around 170, the buffer still has the indoor readings in it. Every outdoor reading then shows up as a spike, when in reality nothing unusual is happening. Once the button is pressed the buffer is emptied, and alerts are held back until enough new readings are collected. It was decided that this should work without the phone. That is why a physical one was used, and it covers the first part of DG3.

D. Cost of the Build

All the parts were bought from shops in Dhaka itself. Table II shows the price of each part in Bangladeshi Taka (BDT). In total the build cost around 4,500 BDT, roughly 37 USD. Commercially sold portable monitors with similar sensors usually go for a few hundred dollars. No custom PCB was needed either.

IV. SOFTWARE AND INTERACTION

Software for the system is split into the firmware on the ESP32-C3, the mobile app and the cloud backend. How these parts connect to each other is shown in Figure 3.

TABLE II
PARTS LIST AND PRICES FOR THE FINAL DEVICE (BDT).

Part	Qty	Price (BDT)
PMS7003 dust sensor	1	2,590
ESP32-C3 SuperMini	1	539
BME280 sensor	1	420
TP4056 charger module	1	32
MT3608 boost converter	1	70
18650 Li-ion battery	1	420
18650 battery holder	1	25
5×7 cm veroboard	1	55
SPDT slide switch	1	5
JST connector	1	20
Capacitors (100 μF, 10 μF)	2	13
Wires, solder, glue, enclosure	-	300
Total		**4,489**

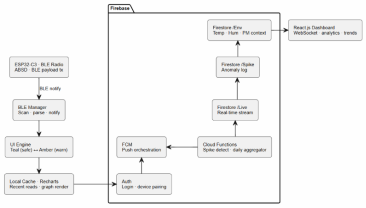


Fig. 3. Overview of the software, with BLE, the mobile app, Firebase and the web dashboard.

A. Firmware and the ABSD Rule

Firmware for the ESP32-C3 was written in C++ on the Arduino framework. The PMS7003 is polled every 10 seconds at 9600 baud over UART, while the BME280 is queried on I2C with the Adafruit driver.

At the core of the firmware sits ABSD. It is how DG1 is met. A circular buffer is used to hold the latest 60 $PM_{2.5}$ values, which works out to roughly 10 minutes of history since one reading comes in every 10 seconds. Each time a new value arrives, the mean and standard deviation of the buffer are worked out again. It was decided that the standard deviation

would never be allowed under 5 μg/m^3. This floor was not part of the plan at the start. During one afternoon of testing the air was so still that the standard deviation became tiny, and a change of just 2 to 3 μg/m^3 was enough to set off an alert. As a result the phone was buzzing every few minutes for no reason at all. The floor was put in after that, and the issue went away.

There are three conditions that a reading has to meet before it counts. First of all, it needs to sit over two standard deviations above the mean. The rise over the mean also has to be larger than 15 μg/m^3. Lastly, the reading should fall in the 1 to 1000 μg/m^3 range, since anything outside of that is most likely a faulty sensor. It has to be stated that the 15 μg/m^3 value was not taken from any reference. Over several days the serial monitor was watched, and the background hardly ever drifted past 10 μg/m^3 unless a clear source was close by, so 15 was chosen based on that. At first glance the validity range might look like it is not needed. It is not. During testing a loose wire made the sensor read exactly zero for several minutes, and those zeros would otherwise have gone straight into the buffer and corrupted the baseline.

Even then, a spike is not confirmed until three or more of the five most recent readings pass every condition. After that, a cooldown of about 10 minutes begins, during which no further alert is sent. Without it, a long episode such as slow traffic would produce a stream of notifications.

Formally, let x_t be the $PM_{2.5}$ reading at step t and let W_t be the N valid readings preceding it. The baseline and its spread are

$$\mu_t = \frac{1}{N} \sum_{x \in W_t} x, \tag{1}$$

$$\sigma_t = \max\left(\sigma_{\min}, \sqrt{\frac{1}{N} \sum_{x \in W_t} (x - \mu_t)^2}\right). \tag{2}$$

A reading is a candidate when

$$c_t = [x_t > \mu_t + k\sigma_t] \wedge [x_t - \mu_t > \Delta_{\min}] \wedge [x_{\min} \le x_t \le x_{\max}], \tag{3}$$

and a spike is declared at step t when

$$s_t = \left[\textstyle\sum_{i=0}^{4} c_{t-i} \ge 3\right] \wedge [t - t_{\text{last}} \ge T_{\text{cool}}], \tag{4}$$

where t_{last} is the step of the last confirmed spike and T_{cool} is counted in steps. Nothing is tested until W_t is full, and the loop as it runs on the device is given in Algorithm 1, and every parameter is listed with its justification in Table III.

None of these parameters is free. If k or $\Delta_{\min}$ is set higher there are fewer alerts, although smaller rises start getting missed. Making the window longer means the baseline is slow to adjust, so going from a quiet lane to a main road would keep looking like a spike for too long. On the other hand if the window is too short the baseline gets lost quickly, and this can happen even before an actual episode is over. The cooldown works the same way, if it is shortened then one long episode can end up sending multiple alerts. The values that were chosen try to balance all of this.

Algorithm 1 Adaptive Baseline Spike Detection (ABSD)

Require: N, k, $\sigma_{\min}$, $\Delta_{\min}$, $x_{\min}$, $x_{\max}$, T_{cool} (Table III)

$W \leftarrow$ empty buffer of size N, $C \leftarrow$ empty buffer of size 5, $t_{\text{last}} \leftarrow -\infty$
loop every 10 s, read x_t from the PMS7003
  if reset button pressed or recalibration write received **then**
    clear W and C ▷ warm-up begins
  end if
  if $x_t \notin [x_{\min}, x_{\max}]$ **then**
    discard x_t, push **false** into C
  else if W is not full **then**
    push x_t into W ▷ still warming up
  else
    $\mu \leftarrow \text{mean}(W)$, $\sigma \leftarrow \max(\sigma_{\min}, \text{sd}(W))$
    $c \leftarrow (x_t > \mu + k\sigma) \wedge (x_t - \mu > \Delta_{\min})$
    push c into C, push x_t into W
    if $\sum C \ge 3$ **and** $t - t_{\text{last}} \ge T_{\text{cool}}$ **then**
      write the current readings to the BLE characteristic
      $t_{\text{last}} \leftarrow t$
    end if
  end if
end loop

TABLE III
PARAMETERS OF ABSD AND THE REASONING FOR EACH VALUE.

Parameter	Value	Reason
Window N	60 readings	About 10 min of readings
Factor k	2	Stays clear of normal noise
Floor $\sigma_{\min}$	5 μg/m^3	No alerts when air is very steady
Minimum rise $\Delta_{\min}$	15 μg/m^3	Drift rarely went past 10 in logs
Valid range	1 to 1000 μg/m^3	Catches faulty output
Debounce	3 of last 5	Filters one-off glitches
Cooldown T_{cool}	60 steps (10 min)	One alert per long event

Once a spike is confirmed and the cooldown is not running, the current sensor values are written to a BLE characteristic and the phone then shows the notification. The reset button works as a hardware interrupt. Pressing it empties the buffer and the running sums, and a warm-up period follows until enough readings have been collected.

B. Phone App

React Native and Expo were chosen for the app so that one codebase could serve both Android and iOS. BLE communication is handled by the react-native-ble-plx library.

Figure 4 shows the main dashboard on the left. The current $PM_{2.5}$ value takes up the top of the screen in a large font. The background is teal by default and switches to amber when a spike arrives, with a short message such as "+40 μg/m^3 above baseline" placed under the number. Below it, a line chart plots

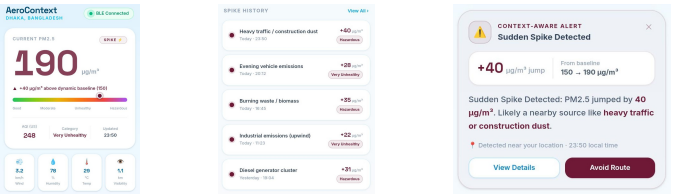


Fig. 4. Screens from the AeroContext app. Left, the main dashboard during a spike. Centre, the spike history. Right, the alert card for one spike.

$PM_{2.5}$ against the running baseline. Further down, small cards carry the temperature, humidity, wind speed and visibility.

Past events can be reviewed on two further screens. This covers the second half of DG3. The history screen lists every spike with its timestamp and the size of the rise (Figure 4, centre). On the alert detail screen (Figure 4, right), a single event is expanded with its figures, a brief explanation and buttons such as "View Details" and "Avoid Route". A recalibration control on the main screen sends the same command over BLE as the physical button.

C. Cloud Storage and Push Alerts

For the database, Firebase Cloud Firestore was used. Every user has three collections in it, for live readings, spike history and the complete environmental log. Alongside the database, two Cloud Functions were set up. Whenever a spike gets written, the first function sends a push notification using Firebase Cloud Messaging, and even if the app is closed the alert still reaches the user. The second runs daily and computes averages from the raw data.

D. Problems During Development

Most of the lost time came from Blynk, which was used early on to test the cloud features. At first every value on the dashboard read as 1. The code was checked. So were the virtual pins and the library version. None of them was the problem. After a few hours it turned out that every datastream maximum in the Blynk console had been set to 1 by default, and every reading was being clamped to it. Finding it was frustrating. This is also part of why Firebase was used in the end.

V. Two-Week Field Test

For two weeks of an ordinary Dhaka summer, the device went everywhere with one of the authors on the daily commute. It rode in a side pocket of a backpack, which left the sensor inlet open to the air. In the first week fixed-threshold mode was used, with an alert for every reading above 150 μg/m^3. In the second week ABSD was switched on, with a baseline reset each morning before leaving home.

The problem with fixed thresholds showed up immediately. Alerts started the minute the user stepped outside and did not stop. Each bus journey brought three or four, crossing the campus on foot brought more, and lunch outside and the trip home added several others. During lectures the buzzing was too much of a distraction and the notifications had to be muted. Before long they stopped being read at all, and were swiped away without a second look.

Midway through the week a heavy rainstorm came out of nowhere and $PM_{2.5}$ dropped quite a lot. However once the rain stopped the numbers climbed back up and all the alerts came back together, which was possibly because of pollutants getting stirred up again after the downpour. By the time the week was over, the device felt more annoying than helpful.

Week two was nothing like the first one. Long stretches went by with the phone completely quiet, even while the dashboard was showing high $PM_{2.5}$. No alerts meant nothing had suddenly changed. That on its own was helpful to know. The first proper spike happened on the third day beside a construction site in Farmgate. The reading went from about 160 up to 215 inside a minute. The phone buzzed only once and showed "+55 μg/m^3 above baseline". There was a visible dust cloud. The road was crossed to avoid it. Later that evening an old bus idling at a junction caused a second one, roughly 35 μg/m^3 above the baseline.

Rain came back on the fifth day. Shortly after it stopped, a brief spike was recorded, most likely from resuspended dust. Relative humidity was over 85% then. The reading could have been inflated by that. It could not be verified. That spike does not help the case for ABSD and it is reported anyway. Only three alerts arrived that whole day. Table IV compares the two weeks.

TABLE IV
How the two alert modes behaved during the test.

	Week 1, fixed threshold	Week 2, ABSD
Alert volume	Dozens a day, three or four on each bus ride	Under twenty across the week
Largest event	Could not be told apart	+55 μg/m^3 near a construction site
User response	Muted in lectures, swiped away unread	Every alert led to some action

Across the whole second week, the log held fewer than twenty spikes. Each one led to some response, whether that meant looking around, moving to the other side of the road or noting a place to avoid. Looking through the history each evening gave a quick record of the worst parts of the day, and that was worth far more than the hundreds of identical warnings of the first week.

This was an early test of the idea ahead of a larger study. The difference was still clear.

VI. Discussion

Fixed thresholds turn into noise almost immediately when the baseline is always high. An alert that depends on what the user has recently been breathing is uncommon, and because it is uncommon it gets noticed. This was achieved by ABSD, which moved with the baseline and fired only when something changed, as DG1 asked for. The cooldown removed the repeated bursts that made the first week so frustrating.

More often than expected, the button was pressed on stepping out of an air-conditioned room. Making it physical was the right call for DG3.

A few limitations have to be stated. Only one user took part, over one season, which is enough for a first test and not for general claims. Spikes were matched to visible sources on the spot rather than to an independent reference, so missed spikes and false alarms were not counted. Humidity correction was not applied either. The alerts also depend on a phone within BLE range. The parameters are fixed in the firmware and there is no proper enclosure yet, since this build concentrated on the alert logic. So whether ABSD carries over to other seasons and other cities is still uncertain.

VII. Conclusion

In this paper, a portable $PM_{2.5}$ monitor was built and carried through Dhaka for two weeks. Its alerts did not use a fixed limit, instead an alert was raised only when a reading jumped sharply above an adaptive baseline.

In spike-detection mode, under twenty alerts reached the user in a week. All of them were read and led to some action. Each one was tied to something happening nearby. Fixed-threshold mode sent hundreds, and most were ignored.

Trials that span several seasons, a correction for humidity and parameters the user can adjust are the next steps. A study with more users comes first. Furthermore, the hardware was chosen to be cheap and easy to build by anyone with a soldering iron, and it was meant for relative spike detection rather than exact readings. For that purpose the current hardware is good enough.

References

[1] World Health Organization, *WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide*, Geneva, 2021.

[2] L. Morawska, P. K. Thai, X. Liu, A. Asumadu-Sakyi, G. A. Ayoko, A. Bartonova, A. Bedini, F. Chai, B. Christensen, M. Dunbabin, J. Gao, G. S. Hagler, R. Jayaratne, P. Kumar, A. K. Lau, P. K. Louie, M. Mazaheri, Z. Ning, N. Motta, B. Mullins, M. M. Rahman, Z. Ristovski, M. Shafiei, D. Tjondronegoro, D. Westerdahl, and R. Williams, "Applications of low-cost sensing technologies for air quality monitoring and exposure assessment," *Environment International*, vol. 116, pp. 286–299, 2018.

[3] M. Badura, P. Batog, A. Drzeniecka-Osiadacz, and P. Modzel, "Optical particulate matter sensors in PM2.5 measurements in atmospheric air," *E3S Web of Conferences*, vol. 44, p. 00006, 2018.

[4] ——, "Evaluation of low-cost sensors for ambient PM2.5 monitoring," *Journal of Sensors*, vol. 2018, p. 5096540, 2018.

[5] B. Alfano, L. Barretta, A. Del Giudice, S. De Vito, G. Di Francia, E. Esposito, F. Formisano, E. Massera, M. L. Miglietta, and T. Polichetti, "A review of low-cost particulate matter sensors from the developers' perspectives," *Sensors*, vol. 20, no. 23, p. 6819, 2020.

[6] V. E. Alvear-Puertas, Y. A. Burbano-Prado, P. D. Rosero-Montalvo, P. Tozun, F. Marcillo, and W. Hernandez, "Smart and portable air-quality monitoring IoT low-cost devices in ibarra city, ecuador," *Sensors*, vol. 22, no. 18, p. 7015, 2022.

[7] D. Suriano, "A portable air quality monitoring unit and a modular, flexible tool for on-field evaluation and calibration of low-cost gas sensors," *HardwareX*, vol. 9, p. e00198, 2021.

[8] S. K. Sharma and A. Kumar, "IoT based air pollution monitoring system using Blynk server," in *2020 IEEE International Conference on Computing, Power and Communication Technologies (GUCON)*, 2020, pp. 667–671.

[9] A. K. M. I. Haque, S. S. Islam, and M. S. Alam, "A low-cost IoT-based air quality monitoring system for developing countries," in *2020 IEEE Region 10 Symposium (TENSYMP)*, 2020, pp. 1554–1557.

[10] S. Palomeque-Mangut, F. Meléndez, J. Gómez-Suárez, S. Frutos-Puerto, P. Arroyo, E. Pinilla-Gil, and J. Lozano, "Wearable system for outdoor air quality monitoring in a WSN with cloud computing: Design, validation and deployment," *Chemosphere*, vol. 307, p. 135948, 2022.

[11] V. Nguyen, "myAQM: Interfacing portable air quality monitor with the Apple watch – An in-the-wild usability study," in *Pervasive Computing Technologies for Healthcare (PervasiveHealth 2023)*, ser. Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, vol. 488. Springer, 2024, pp. 339–363.

[12] J. G. Baseman, D. Revere, I. Painter, M. Toyoji, H. Thiede, and J. Duchin, "Public health communications and alert fatigue," *BMC Health Services Research*, vol. 13, p. 295, 2013.

[13] A. Mehrotra, R. Hendley, and M. Musolesi, "PrefMiner: Mining user's preferences for intelligent mobile notification management," in *Proceedings of the 2016 ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp '16)*, 2016, pp. 1223–1234.

[14] A. M. Becker, H. Marquart, T. Masson, C. Helbig, and U. Schlink, "Impacts of personalized sensor feedback regarding exposure to environmental stressors," *Current Pollution Reports*, vol. 7, pp. 579–593, 2021.

[15] J. A. Robinson, D. Kocman, M. Horvat, and A. Bartonova, "End-user feedback on a low-cost portable air quality sensor system—Are we there yet?" *Sensors*, vol. 18, no. 11, p. 3768, 2018.

[16] S. Bernasconi, A. Angelucci, S. Canali, and A. Aliverti, "Exploring air pollution awareness: insights from interviews with technology enthusiasts using wearable monitoring devices," *Frontiers in Public Health*, vol. 12, p. 1468662, 2024.

[17] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, p. 100, 1954.

[18] S. W. Roberts, "Control chart tests based on geometric moving averages," *Technometrics*, vol. 1, no. 3, pp. 239–250, 1959.

[19] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.